
II. Klient-Server Modelining Dasturiy Realizatsiyasi

Ushbu bo'limda biz Python-ning `socket` moduli orqali operatsion tizim resurslarini boshqarishni va ulanishlar arxitekturasini o'rganamiz.

1. **Server Hayotiy Sikli:** Tizim darajasidagi funksiyalar tahlili (`bind` , `listen` , `accept`).
 2. **Klient Hayotiy Sikli:** Faol ulanish algoritmi (`connect`).
 3. **Ma'lumot Almashish Mexanikasi:** Oqimli ma'lumotlar, kodlash (*UTF* — 8) va segmentatsiya.
-

2.1. Server Hayotiy Sikli (The Server Lifecycle)

Server — bu tarmoqda passiv holatda so'rov kutuvchi sub'ekt. Uning ishlashi uchun operatsion tizim darajasida quyidagi to'rtta fundamental qadam bajarilishi shart:

A. Soket yaratish (`socket.socket`)

Dastur yadrodan (kernel) aloqa uchun **foyli deskriptori** ajratishni so'raydi.

- **AF_INET:** IPv4 protokoli.
- **SOCK_STREAM:** TCP (oqimli) protokoli.

B. Manzilga bog'lash (`bind`)

Bu jarayonda soketga aniq bir **IP-manzil** va **Port** biriktiriladi.

Kiberxavfsizlik eslatmasi: Agar port 0 — 1023 oralig'ida bo'lsa, operatsion tizim administrator (root) huquqini talab qiladi. Shuning uchun biz odatda 1024+ portlardan foydalanamiz.

C. Tinglash rejimi (`listen`)

Soket "passiv" holatga o'tadi. `backlog` parametri navbatda qancha mijoz ulanishini kutishi mumkinligini belgilaydi. Agar navbat to'lib ketsa, yangi mijozlarga `ConnectionRefused` xatosi qaytadi.

D. Ulanishni qabul qilish (`accept`)

Bu **Blocking** (to'xtatib turuvchi) funksiya. Dastur shu qatorda mijoz kelguncha "muzlab" qoladi. Mijoz ulanishi bilan `accept()` ikkita ob'ekt qaytaradi:

1. **Yangi soket ob'ekti (conn):** Aynan shu mijoz bilan gaplashish uchun ochilgan alohida kanal.
 2. **Manzil (addr):** Mijozning IP va Porti.
-

2.2. Klient Hayotiy Sikli (The Client Lifecycle)

Klient faol sub'ekt bo'lib, uning asosiy vazifasi serverning "eshigini taqillatish" hisoblanadi.

- `connect((ip, port))` : Bu funksiya chaqirilishi bilan operatsion tizim darajasida **TCP Three-Way Handshake** boshlanadi. Agar server `listen` holatida bo'lmasa, ulanish darhol rad etiladi.

2.3. Ma'lumot Almashish va Segmentatsiya

TCP ma'lumotni "xabar" sifatida emas, balki **"baytlar oqimi"** sifatida ko'radi.

- `sendall(data)` : Python-dagi eng xavfsiz metod. Agar ma'lumot katta bo'lsa, u barcha baytlar jo'natilmaguncha uzatishni davom ettiradi.
- `recv(buffer_size)` : Bufer hajmi (masalan, 1024 bayt) juda muhim. Agar kelayotgan ma'lumot buferdan katta bo'lsa, u qismlarga bo'linib qoladi.

Marshaling (Kodlash):

$$Bytes = String.encode('utf - 8')$$

Dasturiy darajada biz faqat baytlar bilan ishlaymiz, shuning uchun har bir satr kodlanishi shart.

2.4. Amaliy Realizatsiya (Laboratoriya ishi)

Nazariyani amaliyotda sinash uchun biz eng sodda va ortiqcha "shovqin"lardan xoli bo'lgan kodni tahlil qilamiz. Bu kod soketlarning ishlash mexanizmini tushunish uchun poydevor bo'ladi.

Socket modulining asosiy metodlari

Python `socket` moduli tarmoq ulanishlarini boshqarish uchun quyidagi asosiy metodlardan foydalanadi[cite: 9]:

Metod	Vazifasi
<code>socket()</code>	Yangi soket yaratadi. <code>socket.AF_INET</code> (IPv4) va <code>socket.SOCK_STREAM</code> (TCP) kabi parametrlar bilan ishlaydi.
<code>bind(address)</code>	Soketni muayyan manzil (IP manzil va port) bilan bog'laydi.
<code>listen(backlog)</code>	Soketni tinglash (yangi ulanishlarni kutish) holatiga o'tkazadi.
<code>accept()</code>	Yangi ulanishni qabul qiladi va yangi soket (<code>conn</code>) bilan mijoz manzilini (<code>addr</code>) qaytaradi.
<code>connect(address)</code>	Klient soketini serverga bog'laydi (IP manzil va portga).
<code>send(data)</code>	Soket orqali ma'lumot yuboradi (qisman yuborilishi mumkin).
<code>sendall(data)</code>	Ma'lumotni to'liq yuboradi (yuborilmagan qismlar bo'lsa, davom etadi).
<code>recv(bufsize)</code>	Soketdan ma'lumot qabul qiladi. <code>bufsize</code> – maksimal qabul qilish hajmi.
<code>close()</code>	Soketni yopadi va resurslarni bo'shatadi.
<code>settimeout(value)</code>	Soketning kutish vaqtini belgilaydi (sekundlarda).

Server yaratish metodlari

Server yaratish jarayoni quyidagi metodlarni o'z ichiga oladi:

- **.socket()** : Yangi soket yaratadi. Bu metod server yoki klient uchun tarmoq ulanishi yaratishda asosiy vosita hisoblanadi.
- **.bind()** : Server soketini muayyan IP manzil va port bilan bog'laydi.
- **.listen()** : Serverni ulanishlarni kutish holatiga o'tkazadi. **backlog** parametri yangi ulanishlar uchun navbat uzunligini belgilaydi.
- **.accept()** : Server yangi ulanishni qabul qiladi va u bilan ishlash uchun yangi soket (**conn**) va mijoz manzilini (**addr**) qaytaradi.

Server kodi:

```
import socket

# 1) Soket yaratish
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2) Soketni localhost va port 5000 ga bog'lash
server_socket.bind(("127.0.0.1", 5000))

# 3) Serverni ulanishlarni kutish holatiga o'tkazish
server_socket.listen(5)
print("Server ulanishlarni kutmoqda...")

# 4) Yangi ulanishni qabul qilish
conn, addr = server_socket.accept()
print(f"Yangi mijoz ulanishi: {addr}")

# Mijozdan ma'lumot olish va qaytarish
while True:
    data = conn.recv(1024)
    if not data:
        break
    print(f"Mijozdan keldi: {data.decode('utf-8')}")
    conn.sendall(data) # Ma'lumotni qayta yuborish (echo)

conn.close()
```

Klient yaratish metodlari

Klient server bilan muloqotda quyidagi metodlarni qo'llaydi:

- **.connect()** : Klient soketini serverga bog'lash (IP manzil va portga) uchun ishlatiladi.
- **.sendall()** : Ma'lumotning to'liq yuborilishini ta'minlash uchun ishlatiladi.
- **.recv()** : Klient soketi orqali serverdan ma'lumot qabul qiladi.
- **.close()** : Klient soketini yopish, tarmoq resurslarini bo'shatish uchun ishlatiladi.

Klient kodi:

```
import socket

# 1) Soket yaratish
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 2) Serverga ulanish
client_socket.connect(("127.0.0.1", 5000))

# 3) Ma'lumot yuborish
```

```
client_socket.sendall(b"Salom, server!")
```

```
# 4) Serverdan javob olish
```

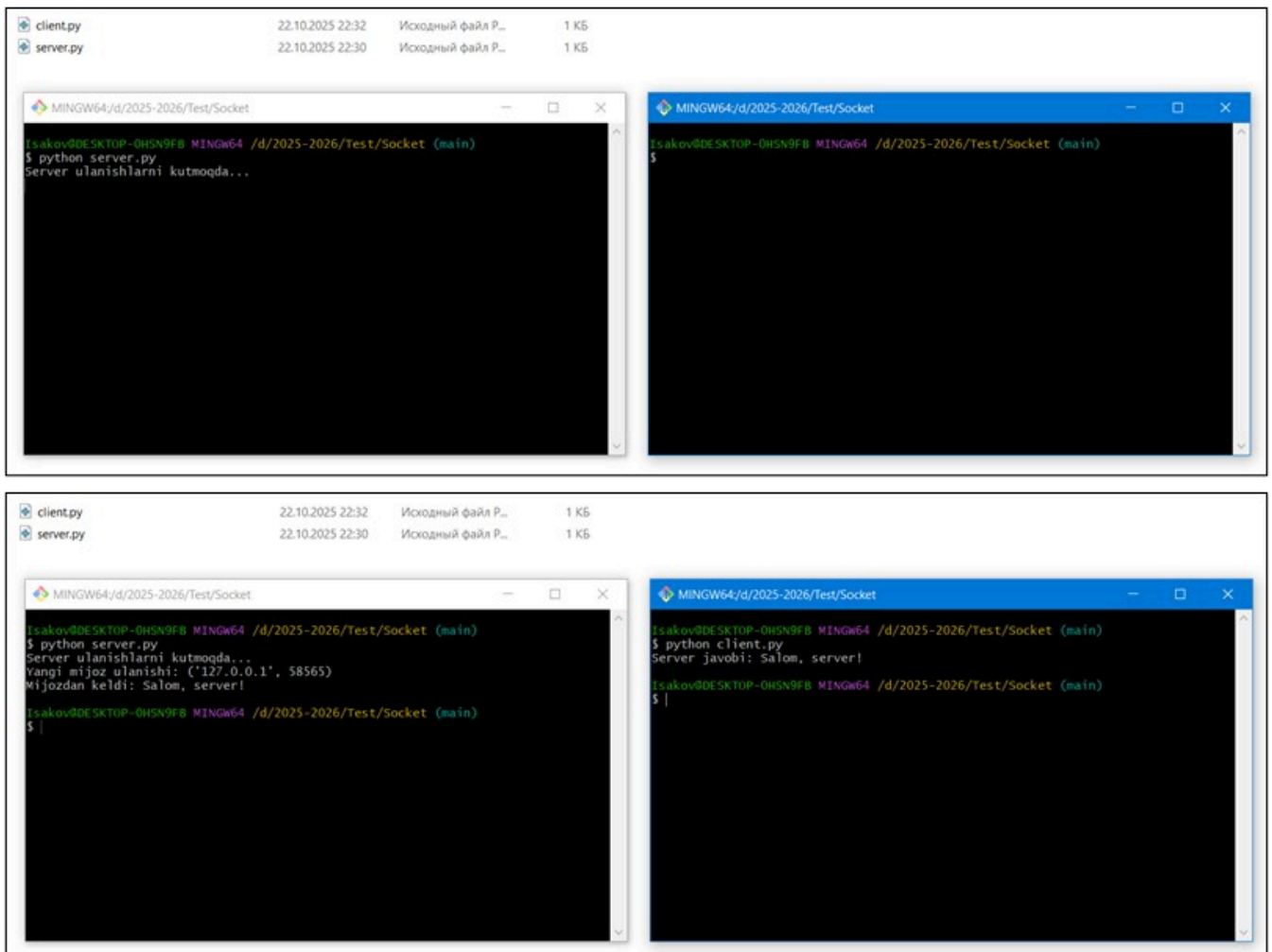
```
response = client_socket.recv(1024)
```

```
print(f"Server javobi: {response.decode('utf-8')}")
```

```
# 5) Soketni yopish
```

```
client_socket.close()
```

Klient va Server muloqotini testlash



Tarmoq muloqoti jarayonida server va klient terminallari orqali ma'lumot almashishni kuzatish mumkin:

Natija tahlili:

- Server 5000-portda tinglaydi, ammo klient har safar ulanish uchun tizim tomonidan tasodifiy portni tanlaydi.
- Server terminalida ulanish manzili va mijoz xabari ko'rinadi.
- Klient terminalida serverdan qaytgan javob (echo) aks etadi.

Xulosa:

- **Klient:** Ma'lumot yuborish uchun `.send()` va javob olish uchun `.recv()` metodlarini ishlatadi.
- **Server:** Ma'lumot qabul qilish uchun `.recv()` va javob yuborish uchun `.send()` metodlarini ishlatadi.
- Har doim aloqa yakunida `.close()` metodini chaqirish resurslarni bo'shatish uchun muhimdir.

